Project - High Level Design

on

AI-Driven Real Estate Educational Quiz Bot

Course Name: Generative AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	Vidhi Yadav	EN22CS3011077
2.	Suhani Pathak	EN22CS301990
3.	Surbhi Jadam	EN22CS3011002
4.	Vanshika Soni	EN22CS3011061
5.	Vinay	EN22CS3011082

Group Name: Group 07D9

Project Number: GAI-07

Industry Mentor Name:

University Mentor Name: Prof. Vineeta Rathore

Academic Year: 2026

Table of Contents

1. **Introduction**.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.

2. **System Design.**
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.

3. **Data Design.**
   3.1. Data Model
   3.2. Data Access Mechanism
   3.3. Data Retention Policies
   3.4. Data Migration

4. **Interfaces**

5. **State and Session Management**

6. **Caching**

7. **Non-Functional Requirements**
   7.1. Security Aspects
   7.2. Performance Aspects

8. **References**

1. Introduction

PropTrain AI is an AI-Driven Real Estate Educational Bot that uses a Retrieval-Augmented Generation (RAG) pipeline and Google Gemini 2.5 Flash LLM to deliver beginner-friendly MCQ-based assessments on real estate property listings. The application is built in Python using Streamlit as the web interface and FAISS as the vector database for semantic retrieval.

1.1 Scope of the Document

This document covers the detailed technical design of the PropTrain AI system. It describes the internal architecture, component-level design, process flows, information flows, API catalogue, UML diagrams, LLM selection rationale, and the overall approach toward solving the problem statement. This document does not cover deployment, infrastructure setup, or future enhancements.

1.2 Intended Audience

Audience	Purpose
Developers	Understand component design and code structure
Evaluators / Faculty	Assess technical depth and design quality
Testers	Understand data flow for test case design
Project Team	Reference for implementation and maintenance

1.3 System Overview

PropTrain AI is a single-user, locally deployed educational bot. The system loads a curated real estate knowledge base, chunks and embeds it into a FAISS vector store, and uses Google Gemini 2.5 Flash to generate MCQ questions and evaluate answers. The Streamlit UI presents questions with 4 clickable options, displays color-coded results with plain-English explanations and real-world examples, and tracks session accuracy. The system requires no database, no backend server, and runs entirely on the free tier of Google Gemini 2.5 Flash (1,500 requests/day at zero cost).

2. System Design

2.1 Application Design

The application is structured as a 3-layer system:

1. **Presentation Layer — app.py**

   Streamlit web application that renders the home screen, quiz screen, MCQ options, evaluation results, and sidebar session stats. It orchestrates all calls to the quiz generator and tutor chat modules and manages all session state.

2. **AI Layer — quiz_generator.py and tutor_chat.py**

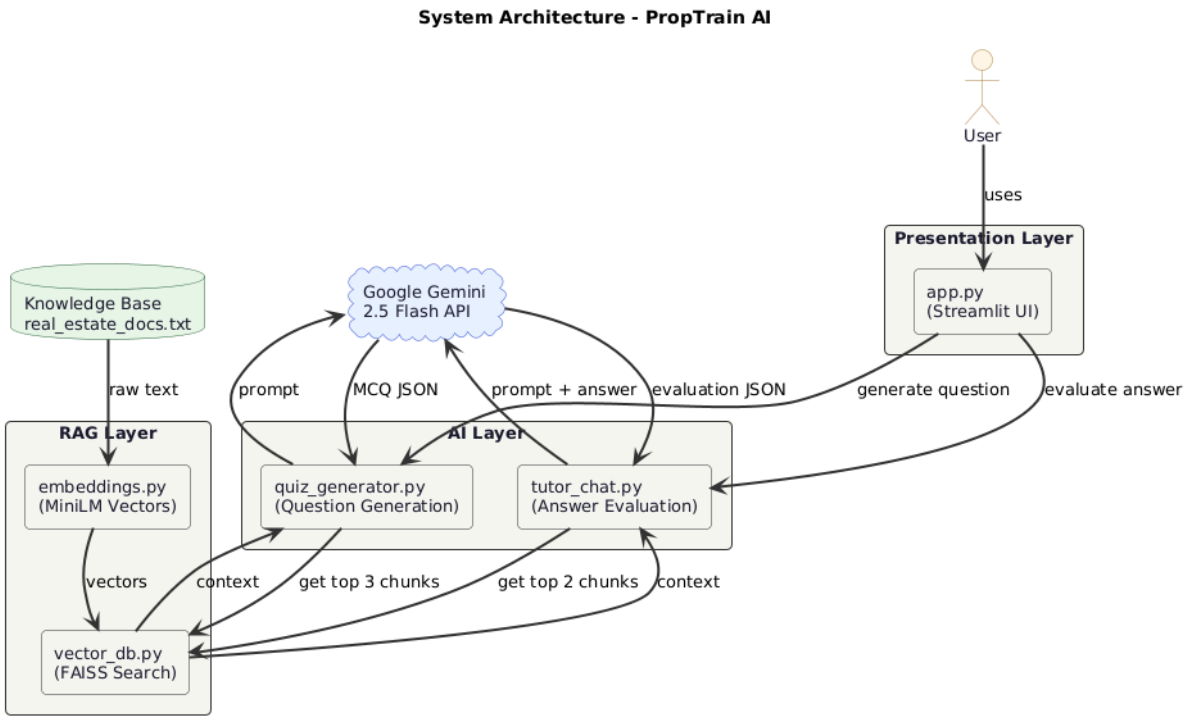
   Two independent modules that communicate with the Google Gemini 2.5 Flash API. quiz_generator.py generates MCQ questions. tutor_chat.py evaluates the user's selected answer and generates detailed explanations with real-world examples.

3. **RAG Layer — embeddings.py and vector_db.py**

   embeddings.py loads and chunks the knowledge base text and generates vector embeddings using sentence-transformers. vector_db.py builds a FAISS index from these embeddings and provides a get_context() method that returns the most semantically relevant chunks for any given query.

4. **Knowledge Base — real_estate_docs.txt**

   A plain text file containing 10 real estate topics with approximately 6,000 words covering MLS terminology, property valuation, cap rates, legal disclosures, closing costs, and more.



2.2 Process Flow

Our app works in **4 simple steps** every time a user opens and uses it:

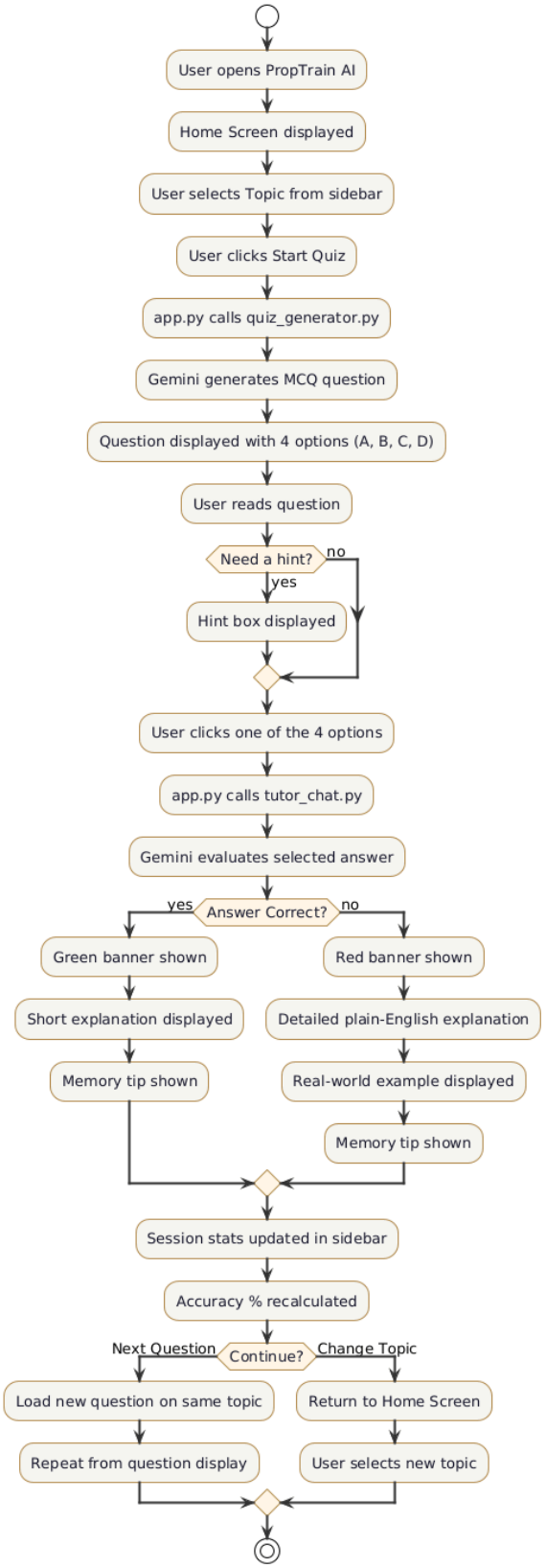
Step 1 — Startup (runs once) When the app opens for the first time, it reads our real estate notes file **real_estate_docs.txt**, breaks it into 31 small chunks, converts each chunk into numbers using the MiniLM model, and stores all of them in FAISS. This happens only once and Streamlit saves it in memory so it does not repeat every time.

Step 2 — Question Generation (every time a user clicks Start Quiz) The user picks a topic from the dropdown and clicks Start Quiz. The app searches FAISS for the 3 most relevant chunks related to that topic and sends them along with a prompt to Google Gemini 2.5 Flash. Gemini reads the context and returns a beginner-friendly MCQ question with 4 options, a correct answer, and a hint — all in JSON format. The question is then displayed on screen.

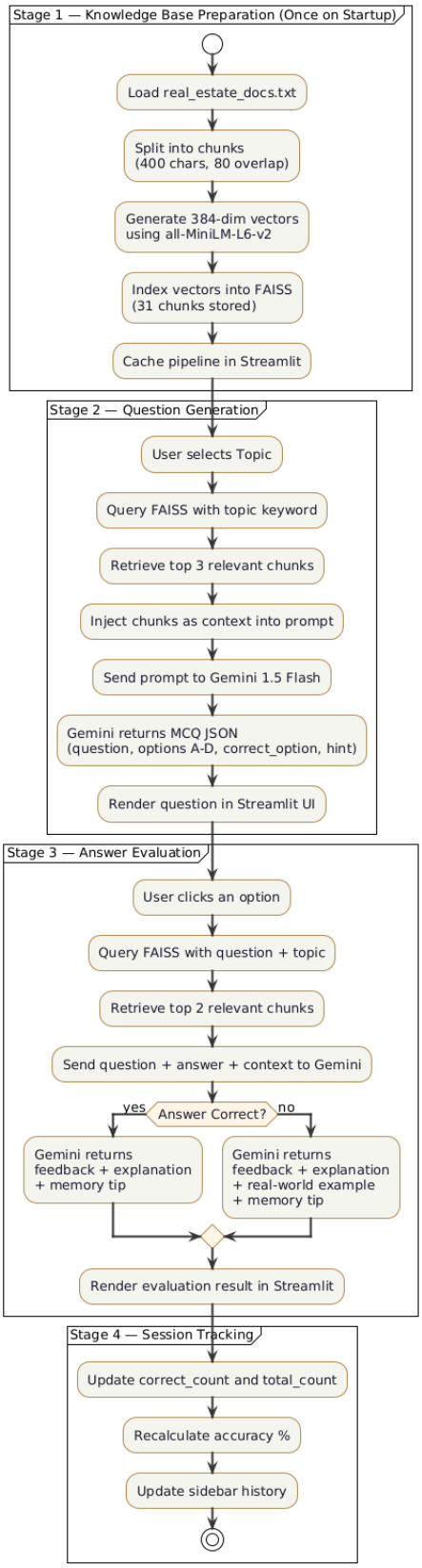
Step 3 — Answer Evaluation (every time a user clicks an option) The user clicks one of the 4 option buttons. The app searches FAISS again for the 2 most relevant chunks and sends the question, all options, the selected answer, and the context to Gemini 2.5 Flash. If the answer is correct, Gemini returns a short explanation and memory tip. If the answer is wrong, Gemini returns a kind message, a detailed plain-English explanation, a real-world everyday example, and a memory tip. The result is shown with green for correct and red for wrong.

Step 4 — Session Tracking (after every answer) After every question, the app adds 1 to the total count and if correct adds 1 to the correct count too. The accuracy percentage is recalculated and the sidebar is updated instantly showing the live score, accuracy, and recent answer history.

PropTrain AI — Application Flow Diagram



PropTrain AI — RAG Pipeline Flowchart



2.3 Information Flow

Information Flow shows **what data moves, from where, and to where** at every step in our app. Think of it like a game of passing the ball — each component receives something, does its job, and passes the result to the next one.

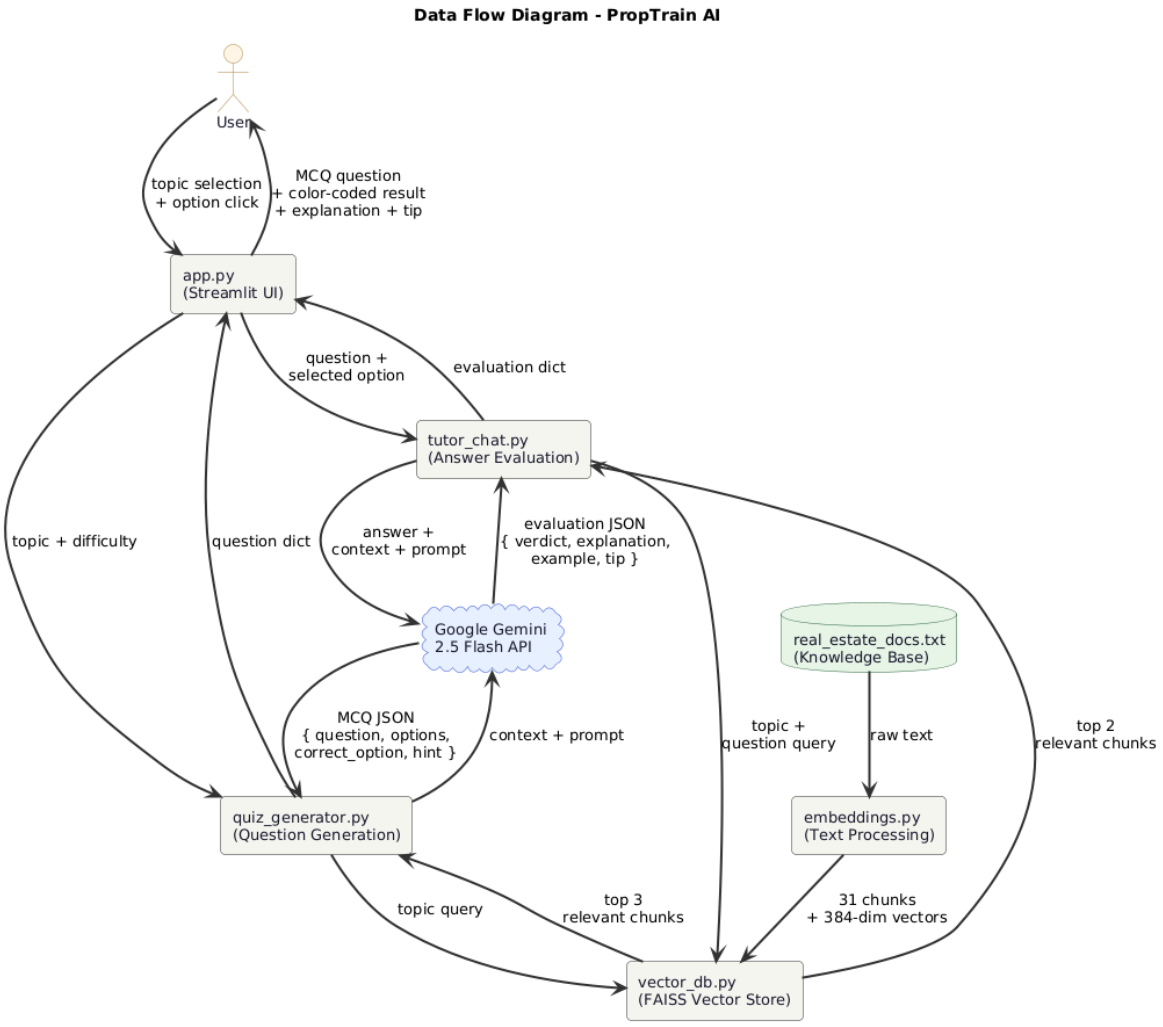
Stage 1 — Knowledge Base to RAG Layer *real_estate_docs.txt* sends raw text to *embeddings.py. embeddings.py* breaks it into 31 chunks and converts them into 384-dimensional number arrays and passes everything to *vector_db.py*. *vector_db.py* stores them in a FAISS index and is now ready to search.

Stage 2 — User to Streamlit The user selects a topic from the dropdown and clicks Start Quiz. This topic string is passed to *app.py* as input.

Stage 3 — Streamlit to Question Generator *app.py* passes the topic and difficulty to *quiz_generator.py. quiz_generator.py* asks *vector_db.py* for the top 3 relevant chunks. Those chunks are combined into a context string and sent as a prompt to Gemini API. Gemini returns a JSON object containing the question, 4 options, correct option, and hint. This is passed back to *app.py* for display.

Stage 4 — User to Tutor Chat The user clicks an option. *app.py* passes the question, all 4 options, the selected option, the correct option, and the topic to *tutor_chat.py*. *tutor_chat.py* asks *vector_db.py* for the top 2 relevant chunks. Those chunks plus the answer are sent to Gemini API. Gemini returns a JSON object with verdict, explanation, real-world example, and memory tip. This is passed back to *app.py* and displayed to the user.

Stage 5 — Result to Screen *app.py* takes the evaluation result and renders it as color-coded HTML — green for correct, red for wrong — along with the explanation box, real-world example box, and memory tip. The session stats are updated in the sidebar instantly.



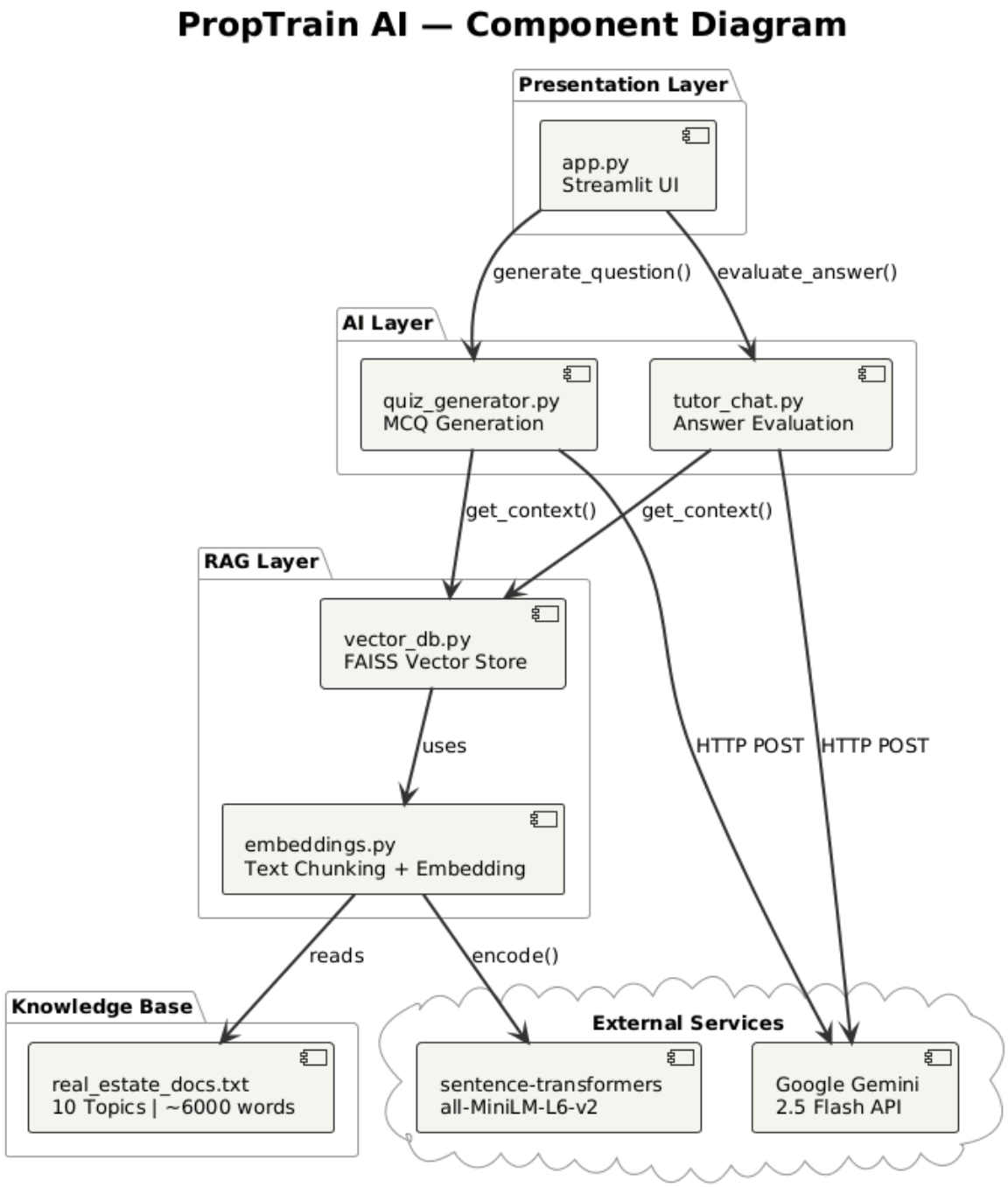
2.4 Components Design

The system is made up of 5 main components. Each component has one clear job and passes data to the next component in the pipeline.

Think of it like a relay race — each component does its part and hands the data to the next one:

- **real_estate_docs.txt** gives raw text to **embeddings.py**
- **embeddings.py** converts that text into numbers (vectors) and gives them to **vector_db.py**
- **vector_db.py** stores those vectors and finds the most relevant ones when asked
- **quiz_generator.py** asks **vector_db.py** for relevant content, then asks Gemini to make a question
- **tutor_chat.py** asks **vector_db.py** for relevant content, then asks Gemini to evaluate the answer
- **app.py** controls everything — talks to both **quiz_generator.py** and **tutor_chat.py** and shows results to the user

Component Diagram:



2.5 Key Design Considerations

These are the important **design decisions** we made while building PropTrain AI and the reason behind each one:

1. Why RAG instead of just using Gemini directly? If we directly ask Gemini a real estate question without any context, it might give wrong or made-up answers. By using RAG, every question and explanation is pulled from our own curated knowledge base. This means the AI never makes things up — all answers are grounded in real content we wrote.

2. Why MCQ format instead of open-ended questions? Open-ended questions are harder to evaluate fairly and can confuse beginners. MCQ with 4 options makes the app accessible to everyone — even people with zero real estate knowledge. It also makes evaluation faster and more consistent.

3. Why two different temperature settings? We use temperature 0.9 for question generation so that every question is different and unique each time. We use temperature 0.3 for answer evaluation so that the feedback is consistent, accurate, and not random. Think of it like this — high temperature means more creative, low temperature means more careful.

4. Why JSON output from Gemini? We need the AI to return structured data — question, options, verdict, explanation — so that our app can display each part separately in the right place on screen. JSON makes this clean and reliable. We also added a safe parser that handles any formatting mistakes Gemini might make.

5. Why Streamlit Session State instead of a database? Since this is a single-user local app, storing data in Streamlit session state is simple, fast, and requires no database setup. The score, history, and question data all live in memory during the session which is enough for our use case.

6. Why Google Gemini 2.5 Flash specifically? It gives high quality responses, supports large context windows, returns reliable JSON, and most importantly — it is available on a free tier which means our app costs absolutely nothing to run.

2.6 API Catalogue

API	Endpoint	Method	Called From	Purpose
Google Gemini	models/gemini-2.5-flash :generateContent	POST	quiz_generator.py	Generate MCQ question
Google Gemini	models/gemini-2.5-flash :generateContent	POST	tutor_chat.py	Evaluate user answer
sentence-transformers	Internal (local model)	—	embeddings.py	Generate text embeddings
FAISS	Internal (in-memory index)	—	vector_db.py	Similarity search

3. Data Design

3.1 Data Model

PropTrain AI follows a **lightweight, in-memory data model** since it is a single-user application with no persistent database.

The system primarily works with **three types of data objects**:

1. Knowledge Chunk Object

Represents a piece of the real estate knowledge base.

Attributes:

- chunk_id — Unique identifier (integer)
- text — Raw chunk content (string)
- embedding — 384-dimensional vector (list of floats)

2. MCQ Object (Generated by LLM)

Represents a quiz question returned by Gemini.

Attributes:

- question — Question text (string)
- options — List of 4 options (array of strings)
- correct_answer — Correct option (string)
- hint — Optional hint (string)

3. Evaluation Object

Represents feedback after user answers.

Attributes:

- is_correct — Boolean (True/False)
- explanation — Plain-English explanation (string)
- real_world_example — Practical example (string)

- memory_tip — Short tip for retention (string)

4. Session State Object (Streamlit)

Maintains runtime data for a user session.

Attributes:

- current_question
- score
- total_attempts
- accuracy
- history (list of past attempts)

3.2 Data Access Mechanism

Data access in PropTrain AI is **fully in-memory and function-driven**, with no external database queries.

Access Flow:

1. Knowledge Retrieval
    - quiz_generator.py and tutor_chat.py call:

      vector_db.get_context(query, k)

    - FAISS performs similarity search using embeddings

2. Embedding Access
    - embeddings are generated once via embeddings.py
    - stored in FAISS index
3. Session Data Access
    - Managed using:

      st.session_state

- Read/write operations happen directly in app.py

Key Characteristics:

- No SQL/No ORM
- Fast retrieval (vector similarity)
- Stateless API calls (Gemini)

3.3 Data Retention Policies

Since PropTrain AI is a **local and stateless application**, data retention is minimal.

Policies:

- Session Data
  - Stored only during runtime
  - Cleared when app reloads or closes
- Knowledge Base
  - Stored as static file (real_estate_docs.txt)
  - Loaded at startup
- FAISS Index
  - Stored in memory (can be optionally persisted)
  - Rebuilt on restart if not cached
- User Data
  - No personal data collected
  - No storage of user identity or inputs

3.4 Data Migration

Currently, **no migration is required**, but future readiness is considered.

Possible Migration Scenarios:

1. Text File → Database
    - Move from .txt to structured DB (MongoDB/PostgreSQL)
2. FAISS → Cloud Vector DB
    - Pinecone / Weaviate / Chroma
3. Local → Multi-user System
    - Add authentication + persistent storage

Migration Strategy:

- Modular design ensures easy replacement of:
    - vector_db.py
    - embeddings.py
- Interface-based retrieval (get_context()) remains unchanged

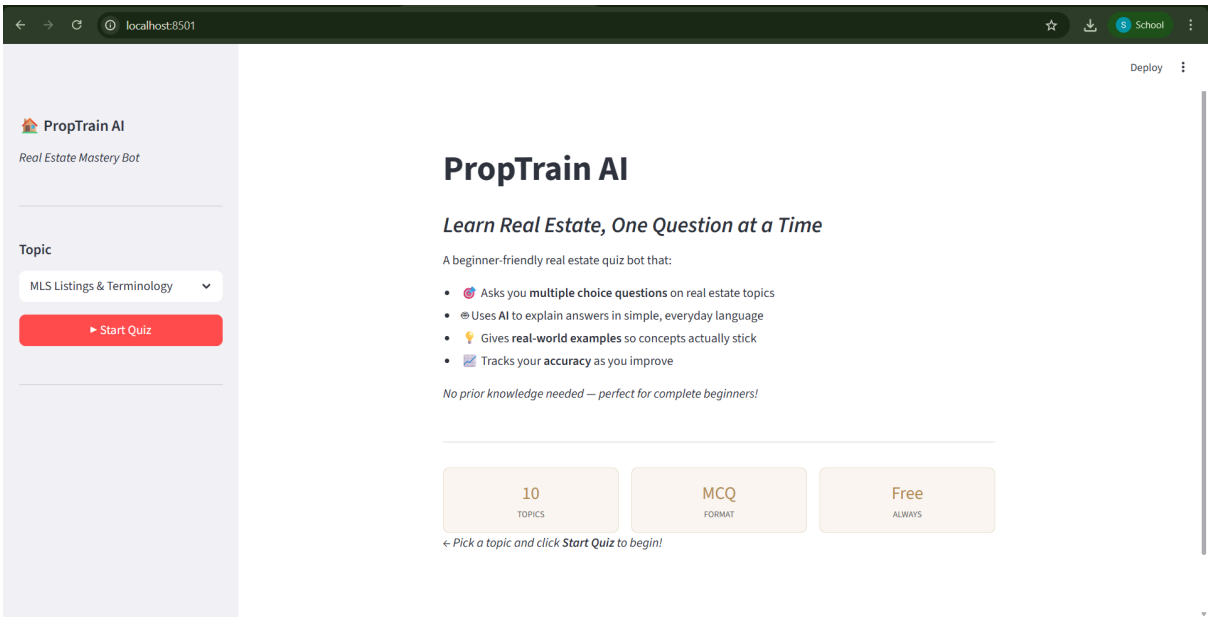
4. Interfaces

4.1 User Interface (UI)

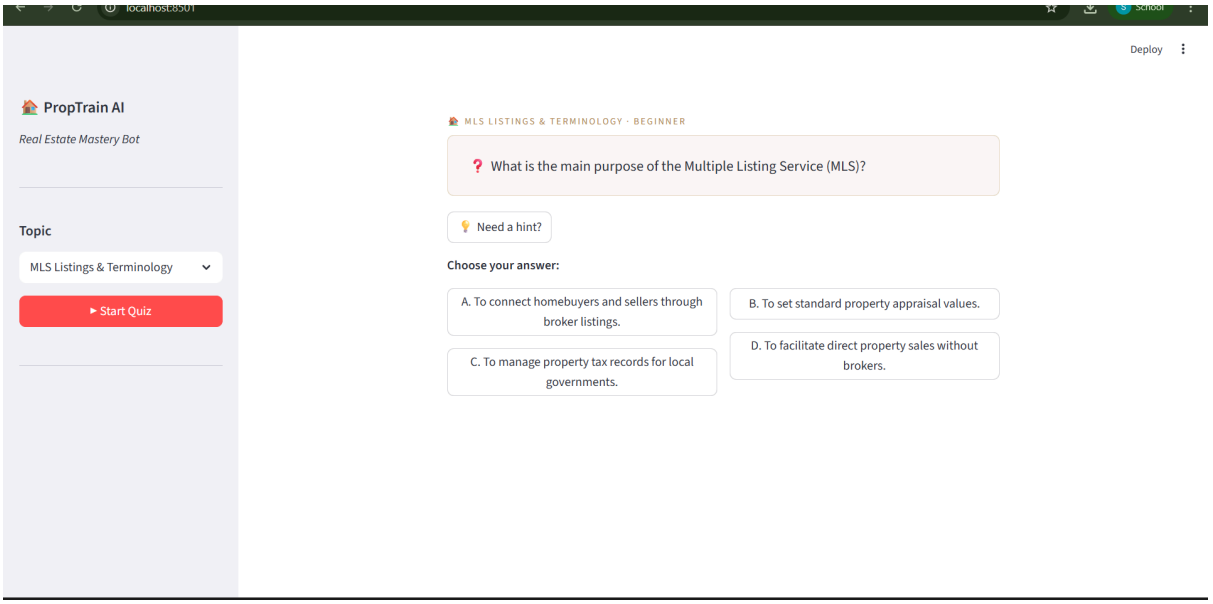
Built using **Streamlit**, consisting of:

Screens:

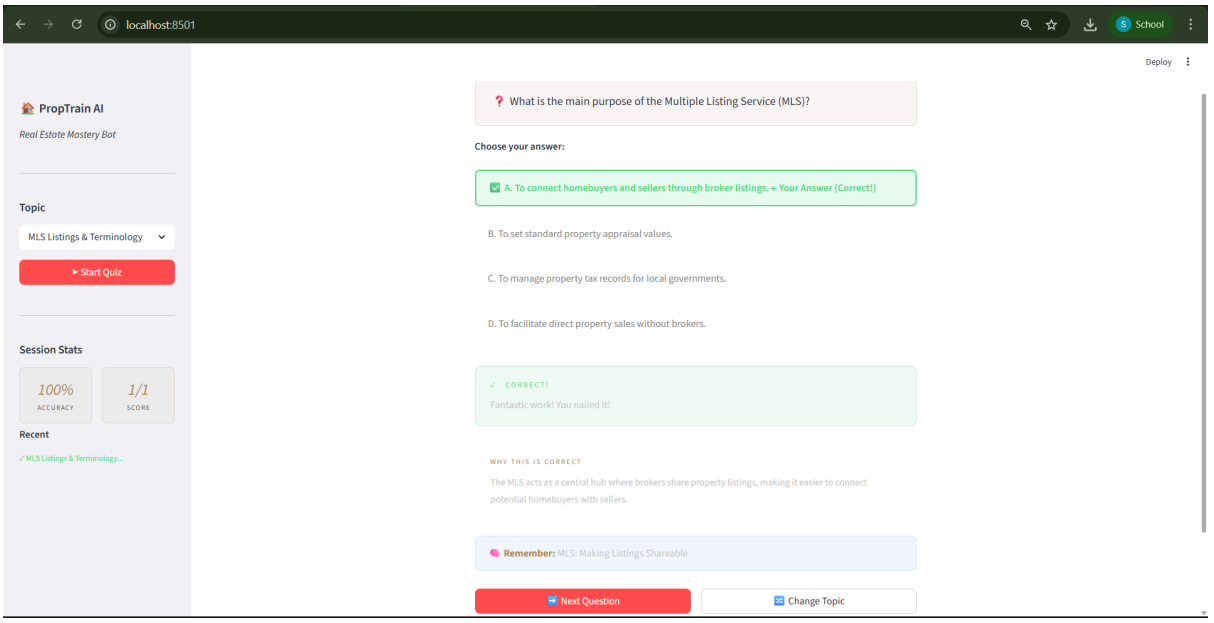
- Home Screen (Topic Selection)



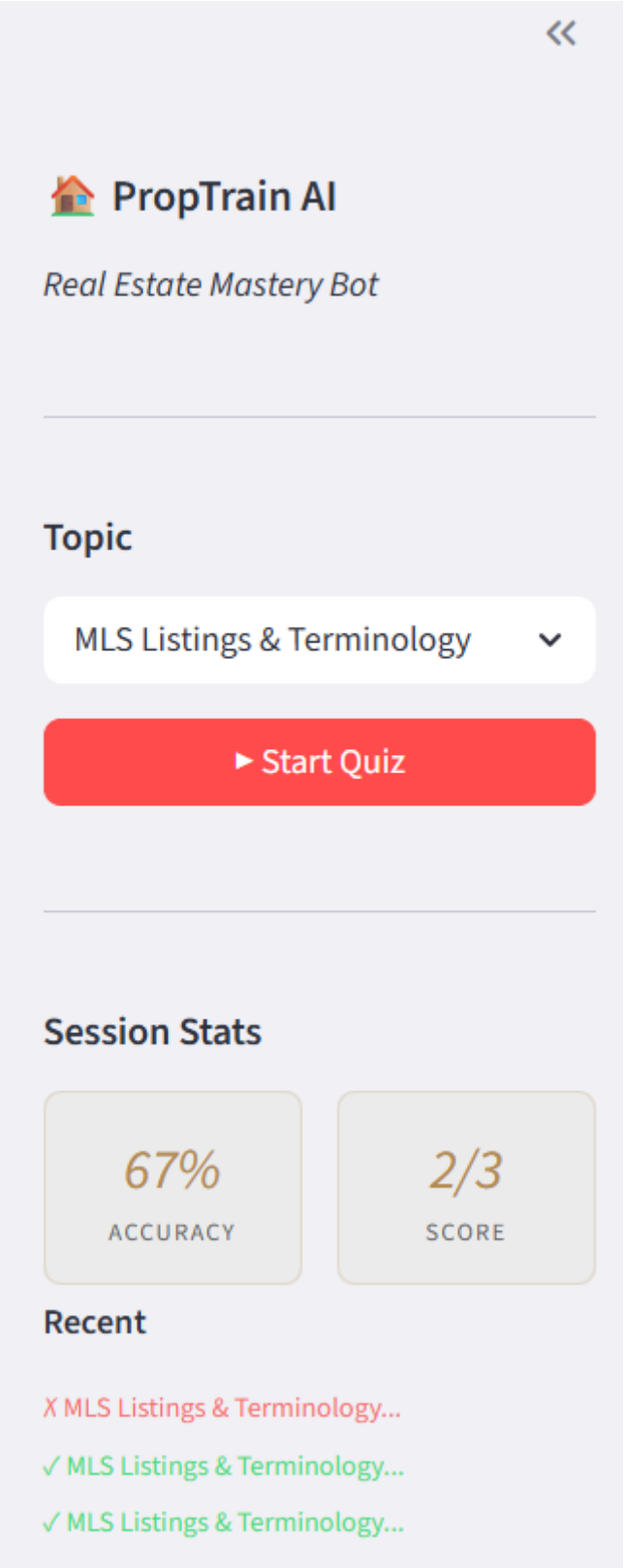
- Quiz Screen (MCQ display)



- Result Display (Explanation + feedback)



- Sidebar (Stats)


PropTrain AI
Real Estate Mastery Bot
Topic
MLS Listings & Terminology
Start Quiz
Session Stats
67%
ACCURACY
2/3
SCORE
Recent
X MLS Listings & Terminology...
✓ MLS Listings & Terminology...
✓ MLS Listings & Terminology...

4.2 Internal Interfaces

1. Quiz Generator Interface

generate_question(topic: str) → MCQ Object

2. Tutor Chat Interface

evaluate_answer(question, options, selected, correct, topic) → Evaluation Object

3. Vector DB Interface

get_context(query: str, k: int) → List[str]

4.3 External Interface

Google Gemini API

- Endpoint: generateContent
- Input: Prompt + Context
- Output: JSON response

5. State and Session Management

PropTrain AI uses **Streamlit Session State** for managing application state.

State Variables:

- st.session_state.score
- st.session_state.total
- st.session_state.current_question
- st.session_state.history

State Flow:

1. Initialize state at app start
2. Update on:

    - Question generation
    - Answer submission
3. Reset when user restarts quiz

Advantages:

- No backend needed
- Instant UI updates
- Simple implementation

6. Caching

Caching is used to **avoid recomputation and improve performance**.

Where Caching is Used:

1. Embeddings + FAISS Index

@st.cache_resource

2. Knowledge Base Loading

@st.cache_data

Benefits:

- Faster startup after first run
- Avoid repeated embedding computation
- Reduced API calls

7. Non-Functional Requirements

7.1 Security Aspects

1. API Key Security

- Stored in environment variables
- Not hardcoded

2. Data Privacy

- No user data stored
- No login required

3. Input Validation

- Sanitize user inputs before sending to LLM
- Prevent malformed prompts

4. Safe JSON Parsing

- Handles invalid LLM outputs gracefully

7.2 Performance Aspects

1. Response Time

- Question generation: ~1–2 seconds
- Evaluation: ~1–2 seconds

2. Optimization Techniques

- FAISS for fast similarity search (O(log n))
- Cached embeddings

- Minimal API calls

3. Bottlenecks

- LLM response latency
- First-time embedding generation

8. References

Reference	URL
Google Gemini API Documentation	https://ai.google.dev/gemini-api/docs
Google GenAI Python SDK	https://github.com/googleapis/python-genai
Sentence Transformers Documentation	https://www.sbert.net
FAISS Documentation	https://faiss.ai
Streamlit Documentation	https://docs.streamlit.io
Gemini Free Tier Rate Limits	https://ai.google.dev/gemini-api/docs/rate-limits
FAISS GitHub Repository	https://github.com/facebookresearch/faiss
PlantUML Documentation	https://plantuml.com